

Soutenance TomTroc - 15 minutes

Projet : TomTroc, plateforme d'echange de livres entre membres.

Positionnement : application web PHP avec architecture MVC, couches entite et manager,

base relationnelle MySQL via PDO, sans librairie PHP externe conformement aux exigences du

projet.

Objectif de la soutenance : presenter le projet tout en montrant un bilan de conformite

honnete par rapport a la fiche d'autoevaluation : ce qui est conforme dans le code, ce qui est

securise, et ce qui reste a valider avec des outils externes.

1. Fil Conducteur De La Presentation

Le message central a faire passer est simple : j'ai realise une application web fonctionnelle qui repond aux attendus structurels du projet, puis j'ai renforce sa conformite technique. Aujourd'hui, les criteres objectivables dans le code sont globalement conformes, notamment l'architecture MVC avec entites et managers, la base relationnelle, le CRUD, la securite de base, Git/GitHub et l'absence de librairie PHP externe. Les points comme W3C, WCAG ou la conformite exacte aux maquettes doivent, eux, etre verifies avec les outils adaptes.

Temps	Partie	Ce qu'il faut dire
0 a 2 min	Contexte et besoin	Presenter TomTroc comme une plateforme de mise en relation entre lecteurs pour partager et echanger des livres, avec inscription, bibliotheque personnelle, profils publics, messagerie et administration.
2 a 5	Architecture	Expliquer le point d'entree unique, le routeur, les controleurs, les vues,

Temps	Partie	Ce qu'il faut dire
min	technique	les entites et les managers. Montrer que le projet respecte l'interdiction des librairies PHP externes imposee par la fiche.
5 a 8 min	Fonctionnalites clés	Montrer les parcours utilisateurs reels : consulter des livres, creer un compte, gerer son profil, ajouter un livre, contacter un membre, acceder a l'administration.
8 a 11 min	Securite et robustesse	Mettre en avant CSRF, mots de passe hashes, requetes preparees PDO, regeneration de session, controles d'accès, validation des uploads et prevention des comportements ambigus.
11 a 13 min	Bilan de conformite	Expliquer ce qui est conforme dans le depot aujourd'hui, puis distinguer les points techniques deja demontres dans le code des points qui demandent encore une validation externe, comme W3C, WCAG ou le controle visuel face a Figma.
13 a 15 min	Conclusion et ouverture	Conclure sur la valeur du projet, les acquis techniques et les evolutions possibles : pagination, logs, tests, politique de roles plus fine, notifications, optimisation de requetes.

2. Script Oral Pret A Dire Avec Reponses Pieges Integrees

Introduction

Bonjour. Je vais vous presenter TomTroc, une application web dediee a l'echange de livres entre membres. L'idee du projet est de permettre a un utilisateur de proposer des livres, de consulter ceux des autres membres, puis d'entrer en contact via une messagerie interne. Mon objectif etait de construire une application complete, claire et maintenable, en respectant le cadre impose par le projet, notamment une architecture MVC, une base relationnelle et l'absence de librairies PHP externes.

Besoin Fonctionnel

Le besoin metier est de faciliter le partage de livres. Pour cela, l'application couvre plusieurs cas d'usage : inscription et connexion, gestion du compte, publication d'un livre, consultation du catalogue, affichage du profil public d'un membre, envoi de messages et administration des contenus et des membres. Cette couverture fonctionnelle est importante car la grille verifie que toutes les pages demandees sont presentes et que tous les liens sont operationnels.

Architecture

Sur le plan technique, le projet repose sur une architecture MVC maison. Le point d'entree HTTP initialise l'application, le routeur associe chaque URL a une action de controleur, les controleurs coordonnent le traitement, les managers dialoguent avec MySQL via PDO, les entites representent les donnees metier, et les vues se concentrent sur le rendu HTML. Cette organisation repond directement a la fiche, qui impose un routeur, des controleurs, des modeles avec une logique entite et manager ou repository, et un systeme de vues evitant la duplication du header et du footer. Si l'on me demande pourquoi ne pas avoir utilise un framework comme Laravel, la bonne reponse est que ce n'etait pas autorise dans ce projet : il fallait justement respecter une implementation sans librairie PHP de ce type.

Choix Techniques

J'ai donc implemente les briques necessaires en PHP : routage, rendu des vues, gestion de session, protection CSRF et helpers d'URL. J'ai egalement structure l'acces aux donnees avec de vraies entites et des managers, tout en gardant des facades de compatibilite pour ne pas casser les fonctionnalites deja en place. L'enjeu n'etait pas de revendiquer un choix personnel de faire sans framework, mais de montrer que je savais respecter les contraintes du cahier d'evaluation. Si l'on me demande pourquoi avoir garde du PHP natif, je peux expliquer que c'etait un attendu explicite du projet, et que cela permet de rendre visibles les mecanismes fondamentaux de l'application.

Fonctionnalites A Montrer

Du point de vue utilisateur, l'application permet de parcourir les livres disponibles, de creer un compte, de gerer un profil, d'ajouter un ouvrage, de consulter le profil public d'un membre et de le contacter par messagerie. Il y a aussi une partie administration pour gerer les livres et les membres. Si le jury demande comment l'application est structuree sans melanger toutes ces fonctionnalites, je peux montrer que chaque parcours passe par un controleur dedie, avec des modeles specialises et des vues separees. Je peux aussi rappeler que la partie admin n'est pas accessible a un utilisateur lambda, ce qui est un point verifie par la fiche.

Parcours Reels Du Site

J'ai egalement formalise les flux reels du site a partir des routes en place. Le parcours principal est le suivant : accueil, catalogue, fiche livre, puis soit connexion ou inscription si l'utilisateur n'est pas authentifie, soit redirection vers la messagerie avec un contexte livre. Cote membre, l'espace Mon compte permet d'editer le profil, d'ajouter, modifier ou supprimer ses propres livres. Cote admin, il existe deux zones distinctes : gestion des livres et gestion des membres. Si l'on me demande

comment je sais que ces flux correspondent au projet reel, je peux m'appuyer directement sur les routes et les controleurs du depot.

Securite

J'ai mis en place plusieurs protections de base. Les mots de passe sont stockes avec **password_hash**, les requetes SQL sont preparees via **PDO**, les formulaires sensibles sont proteges par un token **CSRF**, et la session est regeneree a la connexion. J'ai egalement renforce les validations applicatives, notamment sur l'unicite du pseudo, les uploads d'images et les controles d'accès sur les ressources privees ou administrateur. Si on me demande comment je garantis la securite sans framework, je peux repondre que la securite repose d'abord sur les bonnes pratiques : requetes preparees, hash, validation serveur, sessions et controle d'accès. Si on me demande comment je sais qu'un formulaire vient bien du site, je reponds par le token CSRF verifie cote serveur. Si on me demande pourquoi valider cote serveur alors que certains champs sont deja requis en HTML, je reponds qu'un navigateur ou un client malveillant peut contourner le front, donc la vraie confiance doit rester cote serveur. Enfin, la fiche demande explicitement que les mots de passe soient codes et que les identifiants de connexion a la base ne soient pas visibles sur la derniere version en ligne.

Bilan De Conformite

Si je fais un bilan honnete par rapport a la fiche, plusieurs points sont aujourd'hui objectivement conformes dans le depot. Le README est present. Le projet est bien versionne avec Git et GitHub, avec plus de cinq commits. Le point d'entree unique et le routeur sont en place. Les vues reutilisent un header et un footer communs. La base est relationnelle avec cles etrangeres et suppressions en cascade. Les operations CRUD sont presentes. L'administration est protegee. Les mots de passe sont hashes. Les identifiants sensibles locaux ne sont pas versionnes. Et surtout, l'architecture inclut maintenant de vraies entites et de vrais managers, ce qui repond mieux au critere modele entite ou manager. Si l'on me demande ce que je ne peux pas certifier uniquement en lisant le code, je reponds avec honnetete : la conformite exacte W3C, WCAG niveau 2 et le respect pixel perfect de Figma demandent des outils ou une verification visuelle complementaire.

Ameliorations Recentes A Valoriser

Un travail d'assainissement a ete realise pour rendre le projet plus robuste et plus conforme. J'ai corrige l'ambiguite de connexion liee a l'absence de verification d'unicite du pseudo, alors que la connexion pouvait se faire par pseudo ou email. J'ai fiabilise les uploads pour que les erreurs ne soient plus silencieuses. J'ai retire une modification de schema SQL a chaud dans la gestion des roles admin, ce qui etait risqué en production. J'ai aussi pousse plus loin le respect du MVC en

deplacant du SQL et de la preparation de donnees hors des vues, puis en introduisant une vraie couche entite et manager. Si l'on me demande quel etait le risque principal, je peux citer le pseudo non unique et la mutation du schema a chaud. Si l'on me demande pourquoi du SQL dans un controleur est un probleme, je reponds que cela melange orchestration et acces aux donnees, donc cela nuit a la maintenance et aux tests. Si l'on me demande ce qui prouve que le MVC a ete ameliore, je peux expliquer que les vues affichent maintenant des donnees deja preparees, que le SQL admin est revenu dans les couches de donnees, et que les managers hydratent des entites dediees.

Robustesse Et Limites

Sur la robustesse, le projet gere mieux les comportements degradés qu'avant. Certaines parties du catalogue ont deja des mecanismes de repli, et j'ai renforce des zones qui pouvaient tomber plus brutalement, comme certaines recuperations de livres utilisateur. Si l'on me demande si les uploads sont vraiment securises, je peux repondre qu'ils sont desormais verifies par type MIME, limites a des formats attendus, nommes aleatoirement et qu'ils renvoient une erreur explicite en cas d'echec. Si l'on me demande pourquoi je parle de robustesse sans tests automatises, je peux reconnaitre que les tests sont un axe d'amelioration important, mais que la robustesse passe deja par la clarte des responsabilites, la validation serveur et la suppression des comportements silencieux. Je peux aussi preciser avec honnetete que, selon la fiche, l'absence de responsive correct ne peut pas a elle seule provoquer un refus de soutenance, meme si cela reste souhaitable qualitativement.

Ouverture Et Production

Si le projet devait aller plus loin, mes priorites seraient d'ajouter de vraies migrations de base de donnees, des tests automatisés, une strategie de logs, une meilleure gestion des erreurs, une pagination du catalogue, et une politique de droits plus fine. J'ajouterais aussi une vraie campagne de validation W3C et WCAG avec corrections dediees, afin de transformer les points encore a verifier en points formellement valides. Cela montre que je distingue bien ce qui releve des attendus pedagogiques du projet, de ce qui serait necessaire pour une mise en production plus professionnelle.

Conclusion

Pour conclure, TomTroc est une application complete, fonctionnelle et plus saine apres refactorisation. Ce projet montre ma capacite a repondre a un besoin utilisateur, a structurer une application web, a appliquer les bases de la securite, et a faire evoluer un code existant pour le rendre plus maintenable sans casser l'experience utilisateur. Si le jury me demande quelle est ma plus-value personnelle, je peux dire qu'elle se situe autant dans le developpement que dans la

relecture technique : identifier les risques, corriger sans casser le visuel, et amener le projet a un meilleur niveau de conformite architecturale.

3. Termes Techniques Et Definitions

MVCArchitecture

Modele, Vue, Controleur. Le modele gere les donnees, la vue gere l'affichage, le controleur fait

le lien entre les deux.

EntiteArchitecture

Objet metier representant une donnee du domaine, par exemple un utilisateur, un livre ou un

message.

ManagerArchitecture

Classe chargee des acces aux donnees et de l'hydratation des entites. Elle centralise les

requetes SQL.

RouteurHTTP

Composant qui associe une URL et une methode HTTP, par exemple GET ou POST, a une

action de controleur.

Point d'entree uniqueFront controller

Fichier central, ici *public/index.php*, qui recoit toutes les requetes et delegue ensuite au

routeur.

Autoload[Organisation du code](#)

Mecanisme qui charge automatiquement une classe PHP lorsqu'elle est utilisee, sans avoir a

inclure chaque fichier manuellement.

PDO[Base de donnees](#)

Extension native de PHP pour se connecter a une base de donnees et executer des requetes de

maniere portable et securisee.

Requete preparee[Securite SQL](#)

Requete SQL avec parametres separes des donnees utilisateur, ce qui limite les injections SQL.

Injection SQL[Faille](#)

Attaque consistant a injecter du SQL malveillant dans une requete mal protegee.

CSRF[Securite formulaire](#)

Cross-Site Request Forgery. Attaque qui pousse un utilisateur connecte a soumettre une action

a son insu. Un token unique permet de verifier que le formulaire vient bien du site.

Hash de mot de passe[Securite](#)

Transformation irreversible du mot de passe avant stockage. On ne stocke jamais le mot de

mot de passe en clair.

Session[Etat utilisateur](#)

Mecanisme permettant de conserver l'etat d'un utilisateur connecte entre plusieurs requetes

HTTP.

Regeneration de session[Securite](#)

Changement de l'identifiant de session lors d'une connexion pour limiter les risques de fixation

de session.

Validation serveur[Robustesse](#)

Verification des donnees cote serveur, meme si un controle existe deja cote navigateur. C'est

indispensable car le front peut etre contourne.

MIME type[Upload](#)

Type de contenu d'un fichier, par exemple image/jpeg ou image/png, utilise ici pour filtrer les

uploads autorises.

Refactorisation[Qualite logicielle](#)

Modification de la structure interne du code sans changer le comportement fonctionnel visible.

CRUD[Base de donnees](#)

Create, Read, Update, Delete : creer, lire, modifier et supprimer des donnees en base.

4. Flux A Montrer Au Jury

Flux	Enchainement conforme au projet
Parcours principal	Accueil, catalogue, fiche livre, puis connexion ou inscription si besoin, puis messagerie avec contexte livre.
Authentification	Formulaire, validation serveur, session ouverte, regeneration d'identifiant de session, redirection vers Mon compte.
Espace membre	Mon compte, edition du profil, gestion de la bibliotheque personnelle, suppression de son propre compte.
Messagerie	Premier message autorise via un livre valide ou reponse sur un fil deja existant, puis consultation du fil dans la page Messages.
Administration	Deux pages protegees : gestion des livres et gestion des membres, avec actions en POST pour modifier statuts, roles ou suppressions.
Flux technique	Requete, point d'entree, App, Router, Controller, facade de modele, Manager, Entite, puis rendu de la vue HTML.

Document de flux associe :

Un document dedie reprend ces parcours en detail avec les routes reelles du projet :

docs/FLOWS_SITE.md.

5. Bilan De Conformite A Dire Au Jury

Conforme dans le code :

README present, Git/GitHub avec historique suffisant, routeur et point d'entree unique, vues

avec header et footer communs, base relationnelle avec integrite, CRUD, administration

protegee, mots de passe hashes, aucune librairie PHP externe, architecture avec entites et

managers.

A verifier formellement :

validation W3C, validation WCAG niveau 2 et conformite visuelle exacte aux maquettes Figma.

Ces points demandent des outils ou une verification manuelle et ne doivent pas etre annonces

comme valides si on ne les a pas testes.

Formulation conseillee :

Sur les criteres objectivables dans le code, le projet est globalement conforme. Les points restant

a certifier relevent surtout de la validation externe et de la verification visuelle.

6. Mini Liste De Questions Tres Pieges

- **Le responsive n'est pas parfait, est-ce eliminatoire ?** Non, la fiche precise que cela ne peut pas a lui seul provoquer un refus de soutenance, meme si cela reste un point de qualite.
- **Comment prouvez-vous le critere entite et manager ?** En montrant les classes d'entite dediees et les managers responsables des acces aux donnees, puis les facades de compatibilite qui preservent le comportement existant.
- **Pouvez-vous affirmer que tout est valide W3C et WCAG ?** Non, pas sans passage par les validateurs et les controles adaptes. Je peux dire que le code va dans le bon sens, mais pas certifier un resultat non teste.
- **Comment prouvez-vous que vos flux correspondent au site reel ?** En montrant que chaque etape du parcours correspond a une route definie et a un controleur effectivement present dans le projet.
- **Pourquoi ne pas gerer les roles uniquement en base ?** Parce qu'il faut une source de verite stable et une vraie migration. Le code ne doit pas essayer d'inventer le schema en production.
- **Pourquoi avoir garde du PHP natif alors que cela augmente le travail ?** Parce que c'etait la contrainte du projet, et qu'elle permet de montrer explicitement la comprehension des couches de l'application.
- **Comment savez-vous qu'un formulaire vient bien de votre site ?** Grace au token CSRF compare cote serveur.

- **Pourquoi valider cote serveur si le HTML impose deja required ?** Parce qu'un client peut contourner ou falsifier le formulaire.
- **Qu'est-ce qu'une vue ne devrait idealement pas faire ?** Lancer des requetes, prendre des decisions metier ou transformer lourdement les donnees.

7. Conclusion Courte A Apprendre Par Coeur

TomTroc est une application web complete construite en PHP natif avec une architecture MVC

maison, renforcee par des couches entite et manager. Au-dela des fonctionnalites visibles, j'ai

travaille sur la qualite interne du projet : securite, robustesse, clarte architecturale, conformite a

la fiche et maintenabilite. Cette soutenance montre donc a la fois la capacite a developper une

application fonctionnelle et la capacite a l'ameliorer avec un regard d'ingenierie logicielle.

Document genere pour une soutenance orale d'environ 15 minutes. Conseil : presenter calmement, illustrer avec 2 ou 3 ecrans maximum, et utiliser la mini liste finale uniquement comme pense-bete.